SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
 <input type="text" name="name">
 <input type="email" name="email">
 <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
 <title>Company</title>
 </head>
 <body>
 <div>
 <h1>Welcome
 <p>About us
 </div>
 </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

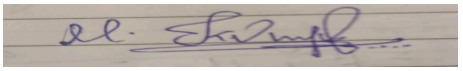
```
<form action="/submit" method="GET">
 <input type="text" name="username">
 <input type="password" name="password">
 <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I KUMARAN M (192525022) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Online Symposium Portal

1. Identify appropriate HTML5 semantic elements

HTML5 semantic elements describe the purpose of different parts of a webpage. The following elements can be used in the symposium portal:

- <header> – Displays the portal title.
- <nav> – Provides navigation links.
- <main> – Contains the main content.
- <section> – Groups related registration content.
- <footer> – Displays copyright information and contact details.

2. Modify the structure to make it standards-compliant

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <title>Symposium Portal</title>
</head>
<body>
    <header>
        <h1>Online Symposium Portal</h1>
    </header>

    <nav>
        <a href="/">Home</a>
    </nav>

    <main>
```

```
        <section>
            <h2>Student Registration</h2>

            <form action="/register" method="POST">
                <label for="name">Name:</label>
                <input type="text" id="name"
                       name="name" required>

                <label for="email">Email:</label>
                <input type="email" id="email"
                       name="email" required>

                <button type="submit">Submit</button>
            </form>
        </section>
    </main>

    <footer>
        <p>Copyright 2026</p>
    </footer>
</body>
</html>
```

This code uses semantic elements, proper labels, required fields, and a submit button.

3. Interpret the HTTP request generated during form submission

When a student clicks the Submit button, the browser sends an HTTP POST request to the server.

Example:

http

```
POST /register HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

name=Kumar&email=kumar%40gmail.com
```

Explanation:

- POST: Sends form data to the server.
- Host: Specifies the server address.
- Content-Type: Specifies the format of the submitted data.
- Request body: Contains the student's name and email.

The POST method sends data in the request body rather than appending it to the URL.

4. Analyze the HTTP response cycle

The HTTP request-response cycle consists of the following steps:

1. Form submission: The student enters their details and clicks Submit.
2. Request generation: The browser creates an HTTP POST request.
3. Server processing: The server validates the submitted information and saves it if valid.
4. Response generation: The server sends an HTTP status code and response content.
5. Browser display: The browser displays the success message or error.

Example HTTP response:

```
http
HTTP/1.1 200 OK
Content-Type: text/html

<h2>Registration Successful!</h2>
```

Common HTTP status codes:

- 200 OK – Request processed successfully.

- 201 Created – Registration record created.
- 400 Bad Request – Request contains invalid data.
- 500 Internal Server Error – Server encountered an error.

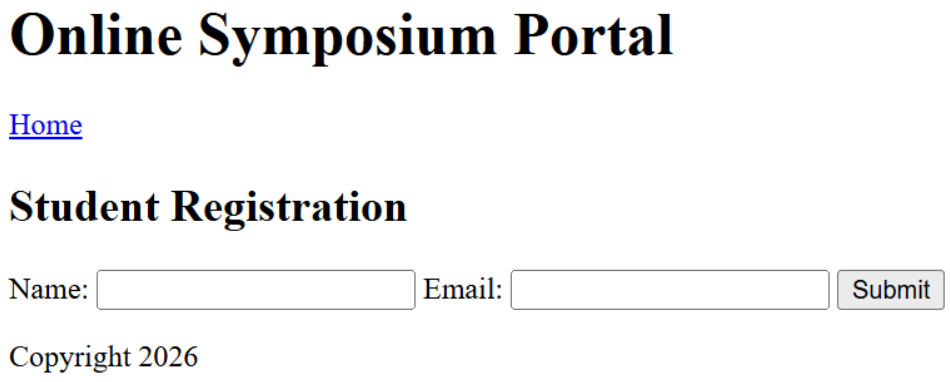
5. Suggest improvements for usability and accessibility

The following improvements make the portal user-friendly and accessible:

1. Labels: Provide labels for all input fields.
2. Validation: Use required and appropriate input types.
3. Error messages: Display clear messages when invalid data is entered.
4. Keyboard accessibility: Allow users to navigate using the keyboard.
5. Responsive design: Support mobile phones, tablets, and computers.
6. Security: Use HTTPS to protect data during transmission.
7. Server-side validation: Validate submitted data on the server.
8. Confirmation: Display a registration success message after successful submission.

Conclusion:

The Online Symposium Portal uses HTML5 semantic elements to organize content and an HTTP POST request to submit student details. Proper response handling, validation, security, and accessibility improve the reliability and usability of the portal.



Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags

The given HTML code contains the following errors:

- The <h1> tag is missing its closing tag </h1>.
- The tag is missing its closing tag .
- The tag is closed, but its child elements are not properly closed.
- The document is missing the <!DOCTYPE html> declaration.
- The <html> tag is missing the lang attribute.

These errors may cause inconsistent webpage layouts across browsers.

2. Analyze how different browsers may interpret this structure

Different browsers may handle malformed HTML differently.

- Some browsers automatically insert missing closing tags.
- Some browsers may interpret the paragraph as part of the heading.
- The page layout and spacing may vary between browsers.
- Browser error recovery may produce unexpected results.

Therefore, valid HTML is necessary for consistent rendering.

3. Identify missing semantic elements

The following HTML5 semantic elements can improve the structure:

- <header> – Contains the website heading.
- <main> – Contains the main page content.
- <section> – Groups related information.
- <footer> – Contains copyright or contact details.

These elements improve readability, accessibility, and document structure.

4. Provide a corrected W3C-compliant HTML structure

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <title>Company</title>
</head>
<body>
    <header>
        <h1>Welcome</h1>
    </header>

    <main>
        <section>
            <h2>About Us</h2>
            <p>Welcome to our company website.</p>
        </section>
    </main>

    <footer>
        <p>Copyright 2026 Company</p>
    </footer>
</body>
</html>
```

Advantages:

- All tags are properly opened and closed.
- Semantic elements organize the content.
- The viewport setting supports mobile devices.
- The document follows standard HTML5 structure.

5. Suggest techniques to ensure cross-browser compatibility

1. Use valid HTML5: Close all tags and maintain proper nesting.
2. Use CSS resets: Reduce differences in default browser styles.
3. Validate HTML and CSS: Use the W3C validators to detect errors.
4. Test multiple browsers: Check Chrome, Firefox, Edge, and Safari.
5. Use responsive design: Apply CSS media queries for different screen sizes.
6. Avoid browser-specific code: Prefer standard HTML, CSS, and JavaScript.
7. Use feature detection: Check whether a browser supports a feature before using it.

Conclusion:

Cross-browser compatibility ensures that a website displays consistently across different browsers. Correct HTML nesting, semantic elements, standards validation, responsive CSS, and browser testing help create a reliable and accessible website.



Form Submission Failure

1. Identify issues in the form configuration

The given HTML form has the following problems:

- The button uses type="button" instead of type="submit".
- The form uses the GET method, which exposes submitted data in the URL.
- The username and password fields do not have required validation.
- There are no labels for the input fields.
- The form does not provide secure handling of login credentials by itself.

2. Analyze why the HTTP request is not triggered

The HTTP request is not triggered because the button is defined as:

HTML

<button type="button">Login</button>

The type="button" element does not submit the form automatically.

Solution: Change it to type="submit" to trigger form submission.

3. Interpret the role of GET method in this scenario

The GET method sends form data as query parameters in the URL.

Example:

http

GET /submit?username=kumar&password=1234 HTTP/1.1

Disadvantages of GET:

- Data appears in the URL.
- Sensitive information may be stored in browser history or server logs.
- URLs may be shared accidentally.
- It is unsuitable for submitting passwords.

Therefore, the POST method should be used for login forms, along with

HTTPS.

4. Propose a corrected implementation

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <title>Login Form</title>
</head>
<body>
    <h2>User Login</h2>

    <form action="/submit" method="POST">
        <label for="username">Username:</label>
        <input type="text" id="username"
               name="username" required>

        <br><br>

        <label for="password">Password:</label>
        <input type="password" id="password"
               name="password" required>

        <br><br>

        <button type="submit">Login</button>
    </form>
```

</body>
</html>
Improvements:

- method="POST" sends data in the request body.
- type="submit" triggers form submission.
- required prevents empty fields from being submitted through normal browser validation.
- <label> improves usability and accessibility.

5. Suggest improvements for secure data transmission

1. Use HTTPS: Encrypts data during transmission.
2. Use POST: Avoids placing credentials in the URL.
3. Validate input: Check submitted data on both client and server.
4. Hash passwords: Store passwords using a secure password-hashing algorithm on the server.
5. Use secure sessions: Protect authenticated sessions with appropriate cookie settings.
6. Display errors safely: Avoid revealing sensitive authentication details.
7. Protect against attacks: Apply rate limiting and other appropriate login protections.

Conclusion:

The main reason for form submission failure is the use of type="button" instead of type="submit". Changing the button type triggers form submission. Using POST, HTTPS, input validation, and secure password handling improves the security and reliability of the application.

User Login

Username:

Password:

Login